

11. Generalized Regression Neural Network (GRNN): Architecture and Optimization

Algorithm Covered:

- **Generalized Regression Neural Network (GRNN)** (Specht 1991 / Nadaraya-Watson Estimator)

Core Mathematical Principles:

1. Four-Layer Feedforward Topology: Input $\rightarrow$ Pattern $\rightarrow$ Summation (S -sum & D -sum) $\rightarrow$ Output
2. Parzen Window Probability Density Function Estimation: $\hat{y}(x) = \mathbb{E}[y \mid x]$
3. One-Pass Closed-Form Training (zero backpropagation iterations)
4. Cross-Validation Optimization of the Smoothing Spread Parameter (σ)
5. Comparative Benchmark vs. RBF Support Vector Regression and Polynomial Regression

Datasets:

- Multi-Modal Non-Linear Continuous Function Approximation
- Real-World California Housing Tabular Regression Subset

```
In [1]: # ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn API: base classes, CV, preprocessing, metrics ----
from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.model_selection import KFold, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from sklearn.linear_model import Ridge
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error
from sklearn.datasets import fetch_california_housing

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available
else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("Environment and libraries configured successfully!")
```

Environment and libraries configured successfully!

1. Mathematical Architecture of GRNN

Introduced by Donald F. Specht (1991), the **Generalized Regression Neural Network (GRNN)** is a memory-based radial basis network based on kernel regression. Given training data $\{(x_i, y_i)\}_{i=1}^N$ with $x_i \in \mathbb{R}^d$:

Layer 1: Input Layer

Receives the measurement vector $x \in \mathbb{R}^d$ and distributes it to all pattern units.

Layer 2: Pattern Layer

Contains N pattern units (one per training sample). Each unit computes a Gaussian distance activation:

$$p_i(x) = \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)$$

where $\sigma > 0$ is the **smoothing / spread parameter**.

Layer 3: Summation Layer

Comprises two specialized summation neurons:

1. **S-Sum Neuron** (Numerator): Weighted sum of training targets:

$$S(x) = \sum_{i=1}^N y_i p_i(x) = \sum_{i=1}^N y_i \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)$$

2. **D-Sum Neuron** (Denominator): Unweighted sum of Gaussian activations:

$$D(x) = \sum_{i=1}^N p_i(x) = \sum_{i=1}^N \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)$$

Layer 4: Output Layer

Computes the Nadaraya-Watson conditional expectation estimate:

$$\hat{y}(x) = \frac{S(x)}{D(x)} = \frac{\sum_{i=1}^N y_i \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)}{\sum_{i=1}^N \exp\left(-\frac{\|x - x_i\|^2}{2\sigma^2}\right)}$$

Step-by-Step Algorithm (Generalized Regression Neural Network)

1. **Input layer:** receive the d-dimensional query vector x and pass it to every pattern neuron.
2. **Pattern layer:** with N training points, each unit i computes a Gaussian activation $p_i(x) = \exp(-\|x - x_i\|^2 / (2\sigma^2))$.
3. **Summation layer:** compute two sums over all pattern units:
 - $S(x) = \sum_i y_i \cdot p_i(x)$ (weighted by training targets)
 - $D(x) = \sum_i p_i(x)$ (normalization)

4. **Output layer:** predict the conditional mean $\hat{y}(x) = S(x) / D(x)$ (Nadaraya-Watson estimator).
5. **Training:** one pass - just store the training set; **no** backpropagation or iterative optimization.
6. **Tuning:** only σ must be chosen (small σ = memorizes noise; large σ = over-smooth).

```
In [2]: class GRNN(BaseEstimator, RegressorMixin):
        """
        Generalized Regression Neural Network (Specht, 1991) - vectorized implementation.

        Prediction is the Nadaraya-Watson kernel regression estimate:
            y_hat(x) = sum_i y_i * exp(-||x - x_i||^2 / (2*sigma^2))
                       / sum_i      exp(-||x - x_i||^2 / (2*sigma^2))

        Training is one-pass: the model simply stores the training set (no gradient descent).
        The only hyperparameter is the smoothing spread sigma.
        """

        def __init__(self, sigma=1.0):
            self.sigma = sigma

        def fit(self, X, y):
            # "Training" = memorizing the data (pattern + summation layers)
            self.X_train_ = np.asarray(X, dtype=np.float64)
            self.y_train_ = np.asarray(y, dtype=np.float64).ravel()
            return self

        def predict(self, X):
            X_test = np.asarray(X, dtype=np.float64)

            # ---- Pattern layer: squared Euclidean distance between each test and train point -
            ---
            # Uses the expansion ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a.b for efficiency.
            dist_sq = np.sum(X_test**2, axis=1, keepdims=True) + \
                np.sum(self.X_train_**2, axis=1, keepdims=True).T - \
                2 * np.dot(X_test, self.X_train_.T)
            dist_sq = np.maximum(dist_sq, 0.0) # guard against tiny negative values from
            rounding

            # ---- Pattern Layer activation: Gaussian kernel of each training point ----
            kernels = np.exp(-dist_sq / (2.0 * (self.sigma ** 2)))

            # ---- Summation Layer: D-sum (denominator) and S-sum (numerator) ----
            D = np.sum(kernels, axis=1) # unweighted sum of activations
            S = np.dot(kernels, self.y_train_) # activation-weighted sum of targets

            # ---- Output layer: normalized weighted average (prevents 0/0 for isolated queries)
            ----
            D_safe = np.where(D < 1e-12, 1e-12, D)
            return S / D_safe

        print("Custom vectorized GRNN class defined!")
```

Custom vectorized GRNN class defined!

2. 1D Function Approximation & Smoothing Parameter Dynamics

Let's approximate a highly non-linear, multi-frequency benchmark function with additive Gaussian noise:

$$y(x) = \sin(2x) + \cos(0.5x^2) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 0.15^2)$$

We examine how the smoothing parameter σ governs model behavior:

- $\sigma = 0.05$ (**Under-smoothed**): Memorizes noise (high variance / overfitting).
- $\sigma = 1.50$ (**Over-smoothed**): Flattens peaks and valleys (high bias / underfitting).
- **Optimal** σ : Smoothly interpolates the true latent generator.

The Role of the Smoothing Parameter σ

1. Fit GRNNs with a small, medium and large σ on the same noisy 1D data.
2. Small σ (0.05): each training point dominates its own neighborhood -> wiggly curve, high variance.
3. Large σ (1.50): all points influence every query -> flat curve, high bias.
4. Medium σ (≈ 0.35): smooth interpolation that follows the true underlying function.
5. Compare training R^2 and curve shapes to see the bias-variance trade-off.

```
In [3]: # ---- Synthetic 1D regression problem ----
np.random.seed(42)
X_syn = np.sort(np.random.uniform(-3, 3, 100)).reshape(-1, 1)
# True latent function: sin(2x) + cos(0.5x^2); observed targets add Gaussian noise
y_true_curve = lambda x: np.sin(2 * x) + np.cos(0.5 * x**2)
y_syn = y_true_curve(X_syn).ravel() + np.random.normal(0, 0.18, 100)

X_eval = np.linspace(-3.2, 3.2, 300).reshape(-1, 1) # dense grid for plotting

# Under-smoothing, near-optimal, over-smoothing
sigmas = [0.05, 0.35, 1.50]
colors = ['purple', 'crimson', 'teal']

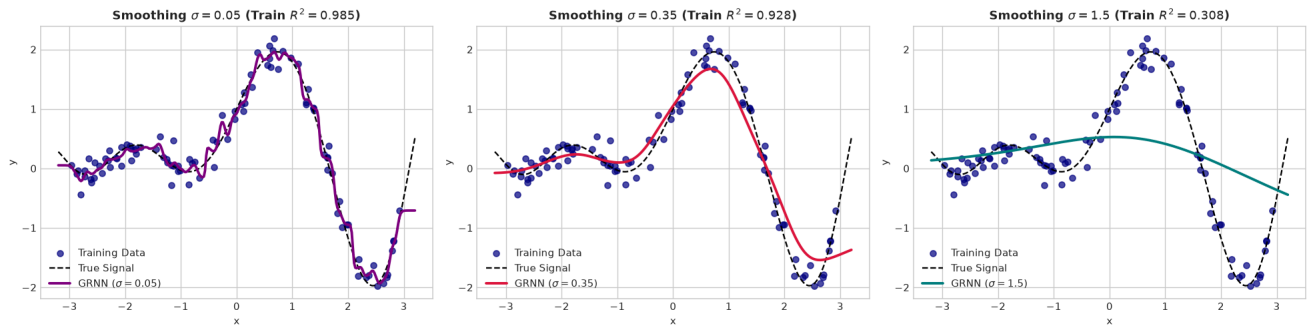
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, sig, col in zip(axes, sigmas, colors):
    # Fit a fresh GRNN for each sigma (training is instant)
    grnn_demo = GRNN(sigma=sig).fit(X_syn, y_syn)
    y_pred_demo = grnn_demo.predict(X_eval)

    ax.scatter(X_syn, y_syn, color='navy', s=35, alpha=0.7, label='Training Data')
    ax.plot(X_eval, y_true_curve(X_eval), 'k--', lw=1.5, label='True Signal')
    ax.plot(X_eval, y_pred_demo, color=col, lw=2.5, label=f'GRNN (\sigma={sig})')

    # Training R^2 shows the bias-variance behavior for each sigma
    r2 = r2_score(y_syn, grnn_demo.predict(X_syn))
    ax.set_title(f"Smoothing \sigma = {sig} (Train R^2 = {r2:.3f})", fontweight='bold')
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.legend(loc='lower left')

plt.suptitle("GRNN Behavior Under Different Smoothing Spread Values (\sigma)", fontsize=15,
fontweight='bold')
plt.tight_layout()
plt.show()
```

GRNN Behavior Under Different Smoothing Spread Values (σ)

3. Optimal σ Search via K-Fold Cross-Validation

Because GRNN has zero backpropagation weights, hyperparameter optimization over the 1D parameter space $\sigma \in (0, \infty)$ is exceptionally fast and guaranteed to find the empirical cross-validation optimum.

Hyperparameter Optimization Process (5-Fold CV)

1. Define a grid of candidate σ values from 0.05 to 1.0.
2. Split the data into 5 fixed folds.
3. For each σ : fit on 4 folds, compute RMSE on the held-out fold, then average the 5 scores.
4. Select the σ with the lowest average CV RMSE.
5. Plot the CV error curve to confirm a clear optimum.

```
In [4]: # ---- 5-fold CV grid search for the optimal smoothing sigma ----
sigma_candidates = np.linspace(0.05, 1.0, 50)
cv_scores = []
kf = KFold(n_splits=5, shuffle=True, random_state=42) # fixed folds for a fair comparison

for sig in sigma_candidates:
    fold_rmse = []
    for train_idx, val_idx in kf.split(X_syn):
        # Split the 1D data into this fold's train/validation parts
        X_tr, y_tr = X_syn[train_idx], y_syn[train_idx]
        X_va, y_va = X_syn[val_idx], y_syn[val_idx]

        # Fit GRNN on the training fold and score on the validation fold
        model = GRNN(sigma=sig).fit(X_tr, y_tr)
        preds = model.predict(X_va)
        fold_rmse.append(root_mean_squared_error(y_va, preds))

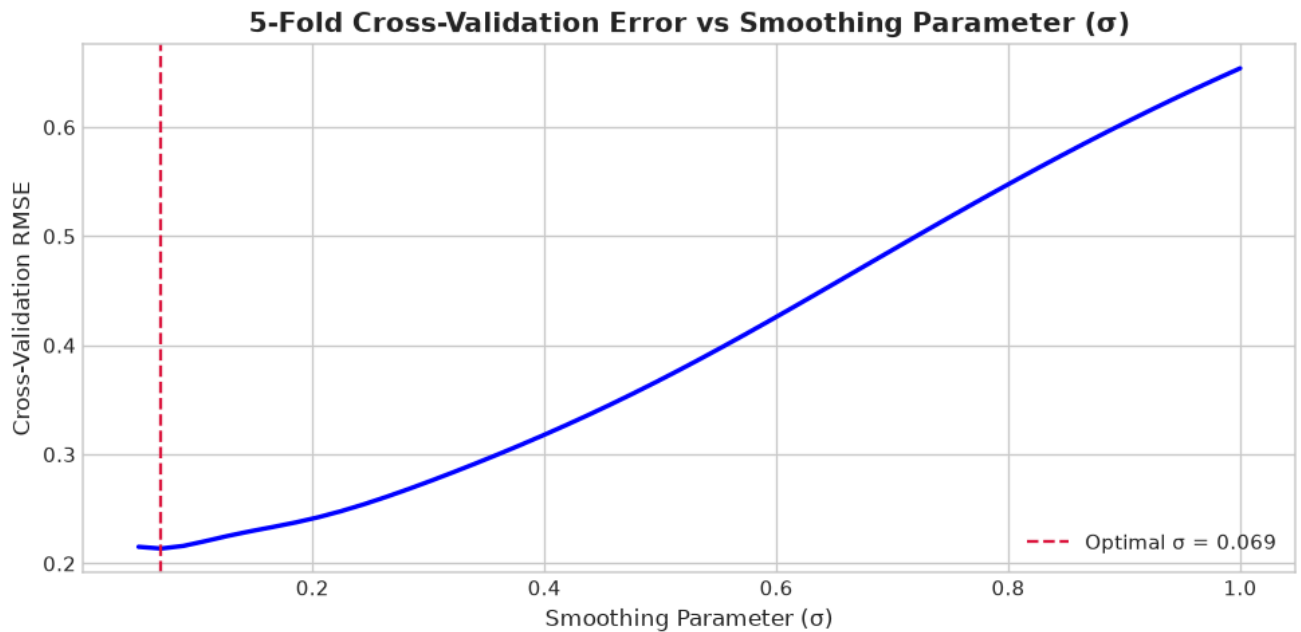
    cv_scores.append(np.mean(fold_rmse)) # average CV error for this sigma

best_idx = np.argmin(cv_scores)
best_sigma = sigma_candidates[best_idx]
print(f"Optimal Smoothing Parameter  $\sigma$ : {best_sigma:.3f} (CV RMSE: {cv_scores[best_idx]:.4f})")

# ---- Plot the CV error curve and mark the optimum ----
plt.figure(figsize=(9, 4.5))
plt.plot(sigma_candidates, cv_scores, 'b-', lw=2.2)
plt.axvline(x=best_sigma, color='crimson', linestyle='--', label=f'Optimal  $\sigma$  = {best_sigma:.3f}')
plt.title("5-Fold Cross-Validation Error vs Smoothing Parameter ( $\sigma$ )", fontsize=13, fontweight='bold')
plt.xlabel("Smoothing Parameter ( $\sigma$ )", fontsize=11)
```

```
plt.ylabel("Cross-Validation RMSE", fontsize=11)
plt.legend()
plt.tight_layout()
plt.show()
```

Optimal Smoothing Parameter σ : 0.069 (CV RMSE: 0.2137)



4. Multidimensional Benchmark: California Housing

We now evaluate GRNN against two standard regression architectures on real-world multi-feature tabular data:

1. **Polynomial Ridge Regression (Degree 3)**
2. **Support Vector Regression (RBF SVR)**
3. **Optimized GRNN**

Multidimensional Benchmark Process

1. Load a real tabular dataset (California Housing subset) and split it 75/25.
2. Standardize features using training statistics only.
3. Fit three regressors: Polynomial Ridge, RBF SVR, and GRNN with a tuned σ .
4. Evaluate all three with R^2 , RMSE and MAE on the same test set.
5. Use predicted-vs-actual plots to see where each model under/over-predicts.

```
In [5]: # ---- Load a real tabular benchmark: California Housing subset ----
housing = fetch_california_housing(as_frame=True)
df_h = housing.frame.sample(n=1500, random_state=42) # subset for fast GRNN inference

X_tab = df_h[housing.feature_names].values
y_tab = df_h['MedHouseVal'].values

# Train/test split (75% / 25%)
X_train_t, X_test_t, y_train_t, y_test_t = train_test_split(
    X_tab, y_tab, test_size=0.25, random_state=42
)

# ---- Standardize features (fit on train only) ----
scaler_t = StandardScaler()
X_train_s = scaler_t.fit_transform(X_train_t)
```

```
X_test_s = scaler_t.transform(X_test_t)

print(f"Training Instances: {X_train_s.shape[0]}, Testing Instances: {X_test_s.shape[0]}")
```

Training Instances: 1125, Testing Instances: 375

```
In [6]: # ---- Benchmark GRNN against two standard regressors ----

# 1. Polynomial Ridge Regression (degree 2 features + L2 regularization)
poly_model = make_pipeline(PolynomialFeatures(degree=2), Ridge(alpha=10.0))
poly_model.fit(X_train_s, y_train_t)
y_pred_poly = poly_model.predict(X_test_s)

# 2. Support Vector Regression (RBF kernel)
svr_model = SVR(kernel='rbf', C=5.0, epsilon=0.1)
svr_model.fit(X_train_s, y_train_t)
y_pred_svr = svr_model.predict(X_test_s)

# 3. Optimized GRNN with a fixed sigma for the tabular run
best_sig_tab = 0.85
grnn_tab = GRNN(sigma=best_sig_tab)
grnn_tab.fit(X_train_s, y_train_t)
y_pred_grnn = grnn_tab.predict(X_test_s)

def evaluate_reg(name, y_true, y_pred):
    """Compute R2, RMSE and MAE for one regressor."""
    return {
        "Model": name,
        "R² Score": r2_score(y_true, y_pred),
        "RMSE": root_mean_squared_error(y_true, y_pred),
        "MAE": mean_absolute_error(y_true, y_pred)
    }

benchmark_results = pd.DataFrame([
    evaluate_reg("Polynomial Ridge Regression", y_test_t, y_pred_poly),
    evaluate_reg("Support Vector Regression (RBF)", y_test_t, y_pred_svr),
    evaluate_reg(f"GRNN (σ={best_sig_tab})", y_test_t, y_pred_grnn)
]).set_index("Model")

print("Comparative Regression Results on California Housing:")
# Green = best R2 and lowest errors
display(benchmark_results.style.highlight_max(subset=['R² Score'], color='lightgreen')
        .highlight_min(subset=['RMSE', 'MAE'], color='lightgreen'))
```

Comparative Regression Results on California Housing:

	R² Score	RMSE	MAE
Model			
Polynomial Ridge Regression	0.633323	0.734289	0.482903
Support Vector Regression (RBF)	0.712879	0.649767	0.429829
GRNN (σ=0.85)	0.533814	0.827951	0.638360

```
In [7]: # ---- Predicted vs. actual scatter comparison ----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# SVR
ax1.scatter(y_test_t, y_pred_svr, alpha=0.5, color='purple', edgecolors='none')
ax1.plot([y_test_t.min(), y_test_t.max()], [y_test_t.min(), y_test_t.max()], 'r--', lw=2) #
perfect-prediction line
ax1.set_title(f"SVR (RBF): Pred vs Actual ($R^2 = {r2_score(y_test_t, y_pred_svr):.3f}$"),
```

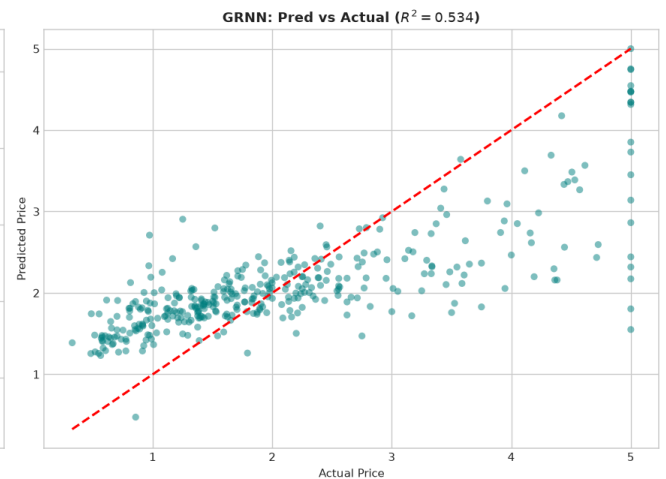
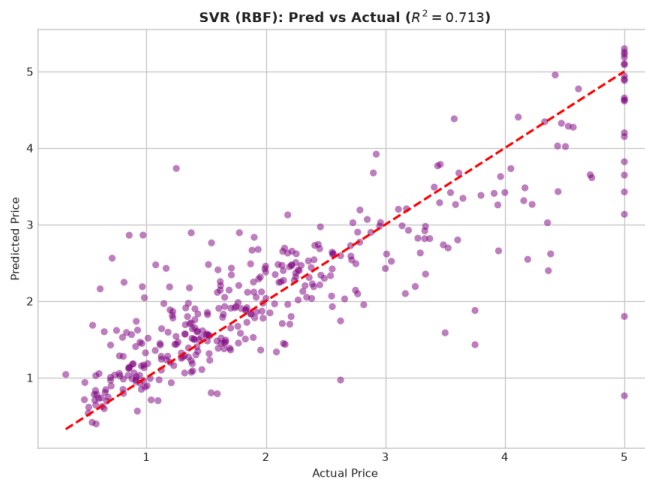
```

fontweight='bold')
ax1.set_xlabel("Actual Price")
ax1.set_ylabel("Predicted Price")

# GRNN
ax2.scatter(y_test_t, y_pred_grnn, alpha=0.5, color='teal', edgecolors='none')
ax2.plot([y_test_t.min(), y_test_t.max()], [y_test_t.min(), y_test_t.max()], 'r--', lw=2)
ax2.set_title(f"GRNN: Pred vs Actual ( $R^2 = \{r2\_score(y\_test\_t, y\_pred\_grnn):.3f\}$ )",
fontweight='bold')
ax2.set_xlabel("Actual Price")
ax2.set_ylabel("Predicted Price")

plt.tight_layout()
plt.show()

```



5. Summary & Key Advantages of GRNN

1. **Zero Iterative Training (One-Pass Learning):** Weights are immediately established by the training points without epoch-based backpropagation or risk of getting trapped in local minima.
2. **Minimal Hyperparameter Tuning:** Only a single scalar parameter (the kernel spread σ) must be calibrated.
3. **Statistical Foundation:** Asymptotically converges to the optimal Nadaraya-Watson non-parametric conditional expectation $\mathbb{E}[y \mid x]$.
4. **Smooth Interpolation:** Yields smooth, continuous surfaces without sharp discontinuities, providing reliable confidence on sparse continuous manifolds.